

Correctr

Engineering Department Doc, Early Foundation Sequence General Developer Handoff Notes

Prepared for downstream Correctr helper chats, Lead Developer review, and project source upload

Project	Correctr
Document type	Engineering Department Handoff Notes
Workstream coverage	Early Foundation Sequence: project setup, hotkey/clipboard proof, basic local correction, structured correction result, manual capture, SQLite event foundation, dictionary event integration
Prepared by role	General Developer
Intended readers	Lead Developer, Project Manager, future developer chats, QA/reviewer chats, future AI/context or memory specialists
Current status	Foundation sequence is handoff-ready after user testing, pending Lead Developer review and next-workstream assignment
Source of truth	Correctr Project Architecture and Workflow Document; current codebase state; user test results from the early implementation sequence

Placement recommendation: Correctr/1.Docs/

1. Purpose and Handoff Statement

This document records the first meaningful engineering foundation sequence for Correctr. It is intended to preserve what was built, why it matters, how it fits the source-of-truth architecture, what files and workflows now exist, what tests passed, and what future chats or developers need to know before continuing.

This is not a final project report and not a user-facing README. It is an engineering handoff document. Future helper chats should use it together with the Correctr Project Architecture and Workflow Document, current repo files, and Lead Developer direction.

- Primary handoff goal: make the early local foundation understandable and reviewable without re-reading the entire chat history.
- Secondary handoff goal: give downstream workers enough context to continue into AI/context correction, personal memory lookup, manual teach-back improvements, or testing cleanup without drifting from the architecture.
- Scope boundary: this document covers completed setup and local foundation work only. It does not claim that Correctr is production-ready or correction-intelligent yet.

2. Current Status at Handoff

Area	Status	Evidence / Notes
Project structure	Working starter repo exists	Main package folder, scripts folder, tests folder, data folder, docs folder, README, requirements, and .gitignore are present.
Hotkey/clipboard loop	Working	User confirmed highlighted text can be copied, processed, and pasted back using Ctrl+Alt+C.
Local dictionary correction	Working but limited	Known typos such as testig, hopfully, maknig, sentences are corrected. Unknown and spacing errors are not.
Structured correction result	Working	CorrectionResult includes original_text, corrected_text, changed, corrections, and engine_version.
Manual correction capture	Working	User manually saved original/corrected examples with source = manual.
SQLite event saving	Working	Dictionary events save from app/hotkey workflow; manual events save from terminal script.
Recent event inspection	Working	scripts/show_recent_events.py displays dictionary and manual correction events.
Automated tests	Working when pytest.ini exists	pytest.ini with testpaths = tests and pythonpath = . is required and should stay in repo.
Next major direction	Pending Lead Developer decision	Likely options: AI/context correction, personal memory lookup, or improved manual teach-back workflow.

3. Source-of-Truth Alignment

The work completed so far follows the architecture sequence: first prove the physical app loop, then add local correction behavior, then structure correction results, then begin local storage for correction history. The project remains local-first and modular.

- The hotkey listener, clipboard handler, correction engine, and database layer are separated by module responsibility.
- The correction engine remains pure. It produces CorrectionResult and does not directly write to SQLite.
- The app workflow bridges the correction engine to database logging for dictionary events.
- Manual correction capture remains separate from automatic dictionary correction.

- No LLM, neural ranker, suggestion UI, tray app, packaging, or advanced spellcheck package has been added yet.
- Personal correction data remains local and protected by .gitignore rules.

4. Focus Area 1 - Setup, Project Structure, and Documentation Foundation

The initial setup created a clean starter structure for Correctr. This matters because future AI helper chats and developers need stable file locations, clear module responsibilities, and predictable testing/documentation conventions.

```
Correctr/
├── 1.Docs/
│   ├── Correctr_Project_Architecture_and_Workflow_Document.docx
│   └── journal.txt
├── correctr/
│   ├── __init__.py
│   ├── clipboard_handler.py
│   ├── config.py
│   ├── correction_engine.py
│   ├── database.py
│   ├── hotkey_runner.py
│   ├── llm_engine.py
│   ├── logging_utils.py
│   ├── neural_ranker.py
│   ├── personal_memory.py
│   ├── space_correction.py
│   ├── spellcheck_engine.py
│   └── suggestion_ui.py
├── data/
│   ├── backups/
│   ├── dictionaries/
│   └── corrections.sqlite
├── scripts/
├── tests/
├── .env.example
├── .gitignore
├── app.py
├── README.md
├── pytest.ini
└── requirements.txt
```

File / folder	Role in current foundation
app.py	Runtime entry point for the hotkey workflow. Now saves changed dictionary events to SQLite.
correctr/config.py	Holds simple runtime settings such as hotkey and timing values.
correctr/hotkey_runner.py	Owns global hotkey listener behavior.
correctr/clipboard_handler.py	Owns copy selected text and paste replacement text behavior.
correctr/correction_engine.py	Owns deterministic local correction and structured CorrectionResult creation.
correctr/database.py	Owns SQLite initialization, save/fetch functions,

	manual correction save, and dictionary event save adapter.
correctr/logging_utils.py	Owns simple non-sensitive console logging.
scripts/	Contains project-useful scripts for init, smoke tests, manual capture, and recent event inspection.
tests/	Contains regression tests for correction engine, database, and event integration.
pytest.ini	Required project test config; keeps local package import working during pytest runs.
.gitignore	Protects private SQLite data, env files, logs, cache files, venvs, and model files.

4.1 Documentation and Package Process Decisions

- Major developer assignments are now expected as zip packages with project-relative paths that can replace or add files cleanly.
- Temporary instruction files should not be included in the repo. Install, unload, and test instructions should be provided in chat unless a long-term project document is intentionally being created.
- Project-useful scripts are allowed in the repo when they support repeatable testing, inspection, initialization, or operation.
- Handoff documents should be created at meaningful checkpoints, not after every tiny edit.
- This document is the first formal engineering handoff for the early foundation sequence.

4.2 Confirmed .gitignore Coverage

The current .gitignore already protects the important local/private files. No change was needed during the database work.

```
# Python
__pycache__/
*.py[co]
.pycache/

# Virtual environments
venv/
.env/

# Environment variables and secrets
.env
.env.*

# Local/private Correctr data
data/*.sqlite
data/*.db
data/backups/
data/exports/
*.sqlite
*.db

# Logs
logs/
*.log
```

```
# Local model files
models/
*.gguf
*.bin
*.safetensors

# OS/editor files
.DS_Store
Thumbs.db
.vscode/
```

5. Focus Area 2 - Hotkey Proof-of-Concept and Basic Correction Logic

The first engineering objective was to prove that Correctr can physically operate in the user workflow. This was the correct first milestone because correction intelligence is not useful if the app cannot reliably copy selected text and paste corrected text back.

Workstream	Result	Notes
Hotkey Clipboard_v0.1	Passed	Ctrl+Alt+C activates Correctr, copies highlighted text, processes it, and pastes replacement text.
Correction Engine_v0.2	Passed	Placeholder prefix was replaced by deterministic local typo replacement.
Correction Engine_v0.3	Passed	Structured CorrectionResult added while preserving a simple app-facing correct_text() function.
Correction Event Integration_v0.1	Passed	Changed dictionary corrections from the app workflow now save to SQLite.

5.1 Hotkey Clipboard_v0.1

- Purpose: prove highlight text -> hotkey -> copy -> Python receives text -> paste replacement.
- Initial placeholder behavior: paste [Correctr test] plus the original selected text.
- User confirmed this worked in a real manual test.
- The initial win was physical workflow reliability, not correction quality.

5.2 Correction Engine_v0.2

- Purpose: replace placeholder output with deterministic local typo replacement.
- Approach: small known-typo dictionary with direct word-level replacements.
- No spelling package, fuzzy matching, spacing logic, LLM, memory, or ranker was added.
- Basic punctuation and capitalization are preserved for simple cases.

Known dictionary examples:

```
testig -> testing
misakes -> mistakes
hopfully -> hopefully
maknig -> making
```

sentences -> sentences
 somehting -> something
 standard -> standard
 nural -> neural
 contexr -> context
 highlight -> highlight
 implimented -> implemented
 enviroment -> environment
 seperate -> separate
 recieve -> receive
 occured -> occurred

5.3 Correction Engine_v0.3

Correction Engine_v0.3 added structured output to prepare for database logging, personal memory, LLM scoring, and future ranker training.

CorrectionResult fields:

- original_text
- corrected_text
- changed
- corrections
- engine_version

Correction record fields:

- original
- corrected
- start_index
- end_index
- reason

- correct_text(text) still returns a plain string for the app workflow.
- correct_text_detailed(text) returns a CorrectionResult.
- The correction engine remains pure and does not write to the database.
- The engine version used in saved events is local_dictionary_v0.3.

5.4 Test Results and Example Behavior

Sample input:

These are testig misakes for the app to hopfully fix. I am maknig two sentences.

Expected output:

These are testing mistakes for the app to hopefully fix. I am making two sentences.

- Smoke test passed.
- pytest passed after pytest.ini was restored.
- Manual hotkey/app tests passed.
- The app/hotkey workflow remains simple from the user perspective: highlight text, press Ctrl+Alt+C, receive corrected text.

6. Focus Area 3 - SQLite Database Setup, Manual Capture, and Event Integration

The database foundation was added because Correctr needs local correction history before personal memory, LLM evaluation, and neural-ranker training can become meaningful. The most important concept is that the useful datapoint is the original bad text paired with the final corrected text.

6.1 Database Schema

Table: correction_events

Fields:

- id
- created_at
- source
- original_text
- corrected_text
- changed
- corrections_json
- engine_version
- notes

Indexes:

- idx_correction_events_created_at
- idx_correction_events_source

- The schema is intentionally simple but future-aware.
- corrections_json stores structured correction records as JSON text.
- source separates dictionary corrections, manual corrections, and future correction sources.
- The current allowed sources are dictionary, manual, future_llm, future_memory, and future_ranker.

6.2 Database Functions Added

```
get_database_path()
initialize_database()
save_correction_event()
save_dictionary_correction_result()
save_manual_correction()
fetch_recent_correction_events()
```

- initialize_database() creates data/corrections.sqlite and the correction_events table if needed.
- save_correction_event() is the general event insertion function.
- save_manual_correction() stores manual teach-back examples as source = manual.
- save_dictionary_correction_result() stores changed CorrectionResult objects as source = dictionary.
- fetch_recent_correction_events() powers recent-event inspection and test assertions.

6.3 Manual Correction Capture_v0.1

- scripts/manual_correction_capture.py lets the user enter original bad text, corrected text, and an optional note.
- The script saves a manual event with source = manual and engine_version = manual_capture_v0.1.
- This captures failed or incomplete corrections as useful future training data.
- The current script is terminal-based and single-line input only.

Manual capture example:

Original: shiuld this not also be abl eto fix this?

Corrected: Should this not also be able to fix this?
Source: manual

6.4 Correction Event Integration_v0.1

- app.py now calls correct_text_detailed() so it can access CorrectionResult metadata.
- The app still pastes only corrected_text back to the user.
- If result.changed is true, the app saves a dictionary correction event to SQLite.
- If result.changed is false, no dictionary event is saved in v0.1.
- No-change events are skipped to avoid filling the database with low-value no-op attempts before a clearer event policy exists.

6.5 Confirmed Saved Events

The user ran scripts/show_recent_events.py and confirmed both dictionary and manual events are saved.

Confirmed saved events:

ID 3 - source = dictionary

ID 2 - source = manual

ID 1 - source = manual

Dictionary event confirmed fields:

- original_text

- corrected_text

- source = dictionary

- changed = True

- engine_version = local_dictionary_v0.3

- structured correction records

Manual event confirmed fields:

- original_text

- corrected_text

- source = manual

- changed = True

- engine_version = manual_capture_v0.1

Important limitation shown by saved events:

Input:

These are testig mis takes for the app to hopfully fix. i ma maknig two sentencs.

Dictionary output:

These are testing mis takes for the app to hopefully fix. i ma making two sentences.

Manual corrected version:

These are testing mistakes for the app to hopefully fix. I am making two sentences.

Meaning:

The dictionary engine fixed known word typos but did not fix spacing/context errors like mis takes -> mistakes or i ma -> I am.

7. Tests, Scripts, and Validation Commands

Command / script	Purpose	Current status
python scripts/init_database.py	Initialize local SQLite database and schema.	Worked.
python scripts/manual_correction_capture.py	Capture manual original/corrected training example.	Worked.
python scripts/smoke_test_correction.py	Verify local correction engine output without hotkey app.	Worked.
python scripts/show_recent_events.py	Inspect recent SQLite events.	Worked.
pytest	Run correction, database, and integration tests.	Worked when pytest.ini exists.
python app.py	Run Correctr hotkey app.	Worked in manual testing.

7.1 Test Coverage Added

- Correction engine tests cover sample sentence correction, no-change sentences, capitalization, punctuation, multiple replacements, expanded dictionary entries, plain-string `correct_text()`, and structured `CorrectionResult` output.
- Database tests cover schema initialization, manual save, fetch, source storage, corrections_json, changed flag inference, newest-first event order, invalid source validation, and limit validation.
- Correction event integration tests cover saving `CorrectionResult` as dictionary source, original/corrected text persistence, correction records, engine version, no-change skip policy, and manual save compatibility.

7.2 pytest.ini Requirement

pytest.ini is a real project config file and should stay in the repo. Without it, pytest may fail with `ModuleNotFoundError: No module named correctr`.

```
[pytest]
testpaths = tests
pythonpath = .
```

8. Decisions Made During This Foundation Sequence

Decision	Reason	Impact
Use zip assignment packages for major tasks	Keeps implementation work grouped, reviewable, and easy to unload into the repo.	Future developer outputs should include project-relative paths and no clutter files.
Do not include temporary instruction docs	Temporary docs clutter the repo and are not useful long-term.	Unload/test instructions should be in chat unless creating a real project document.
Keep correction engine pure	Correction logic should not own persistence side effects.	app.py or small adapter functions decide when to save events.
Skip no-change dictionary events for now	Avoid database noise before a formal event policy exists.	Only changed dictionary results are currently saved from the app

		workflow.
Use SQLite with JSON correction records	Simple local-first storage fits the project and preserves structured data.	Future memory/ranker/LLM work can query the same table.
Keep manual capture terminal-based for v0.1	Enough to capture training examples without building UI early.	Cleaner UI/manual teach-back can be later.
Keep pytest.ini in repo	Required for stable local imports during pytest.	Do not delete it.

9. Assumptions, Known Issues, and Limitations

9.1 Assumptions

- The user is running on Windows through VS Code and PowerShell.
- The project root is currently named Corectr locally, even though the project name is Correctr.
- The default database path is data/corrections.sqlite.
- The current correction engine intentionally handles only exact known typo dictionary matches.
- Manual corrections are trusted user-provided examples.
- Private correction data should not be committed or uploaded publicly.

9.2 Known Limitations

- The dictionary engine does not fix spacing mistakes like abl eto -> able to or mis takes -> mistakes.
- The dictionary engine does not fix contextual/capitalization phrase issues like i ma -> I am.
- The app does not yet provide accept/reject/edit UI after an automatic correction.
- Manual capture is separate from the hotkey workflow and requires running a terminal script.
- show_recent_events.py prints saved text, so it should remain local-only.
- The database currently has no migrations system; schema changes should be cautious.
- No confidence scoring exists yet.
- No personal memory lookup uses saved manual corrections yet.
- No LLM/context layer exists yet.

9.3 Non-Issue Clarification: Newline After Paste

A suspected extra-newline paste issue was later identified as likely user selection behavior: triple-clicking selected the full line including the trailing newline. This should not be treated as a confirmed app bug. A future setting could optionally strip one trailing newline from single-line selections, but that should be deliberate because multi-line selections may need to preserve line breaks.

10. Downstream Guidance for Future Chats and Developers

10.1 What Future Workers Should Read First

1. Correctr Project Architecture and Workflow Document.
2. This handoff document.
3. Current app.py.
4. correctr/correction_engine.py.

5. correctr/database.py.
6. scripts/show_recent_events.py output if the user provides it.
7. Current pytest results.

10.2 What Future Workers Should Not Assume

- Do not assume the correction engine is good enough for real user mistakes.
- Do not assume saved manual corrections are already being used by the correction engine.
- Do not assume no-change events are saved.
- Do not assume the database schema is final.
- Do not assume the app has any UI or accept/reject flow.
- Do not assume AI/LLM logic exists yet.
- Do not log or expose full personal correction text unless needed for local debugging or explicit user review.

10.3 Recommended Next Workstream Options

Option	Why it matters	Caution
AI/Context Correction_v0.1	Needed because current dictionary logic cannot fix realistic errors like shiuld, abl eto, i ma, or sentence capitalization.	Must be controlled, privacy-aware, and should preserve user style. Do not rewrite broadly.
Personal Memory Lookup_v0.1	Could use saved manual corrections to improve repeated mistakes before calling AI.	Needs clear matching policy so it does not over-apply old corrections.
Manual Teach-Back Workflow_v0.2	Would make failed correction capture smoother and more directly connected to the app workflow.	Avoid building a polished UI too early.
Testing/Decision Log Cleanup	Would lock in early dependency and schema decisions before a more complex AI layer.	Do not spend too long documenting without building.

General Developer recommendation: the next major technical direction should likely be AI/Context Correction_v0.1 or a tightly scoped manual teach-back improvement. The saved events demonstrate that dictionary-only correction is already too weak for the user's real typing patterns.

11. What the Lead Developer Should Review

- Confirm whether the current correction_events schema is acceptable for the next phase.
- Confirm whether no-change dictionary events should continue to be skipped.
- Confirm whether save_dictionary_correction_result() belongs in database.py long-term or should move into a future orchestrator module.
- Confirm whether app.py should remain the integration point or whether a correction orchestration layer should be introduced before AI/context work.
- Confirm the next workstream: AI/context correction, memory lookup, manual teach-back, or cleanup/testing docs.
- Verify that pytest.ini, .gitignore, data privacy rules, and scripts are all acceptable project-level decisions.

12. What Another Developer Needs to Continue

- Run pytest before and after any changes.
- Use temporary database paths in tests. Do not write tests against data/corrections.sqlite.
- Do not include corrections.sqlite in assignment packages.
- Do not include temporary instruction files in repo packages.
- Do not add dependencies unless the Lead Developer approves the need.
- Keep correction_engine.py pure unless a deliberate architecture change is approved.
- Use scripts/show_recent_events.py to inspect saved local events when debugging.
- Keep manual corrections and dictionary corrections clearly separated by source.
- Treat saved correction data as private local data.

13. Handoff Summary

The early Correctr foundation now works end-to-end at a local level. The app can correct known typos from highlighted text, paste the corrected result back, save dictionary correction events, capture manual corrections, and inspect recent SQLite events. The project is not yet intelligent enough for the user's real typo patterns, but it now has the necessary local foundation to begin AI/context correction, personal memory lookup, or improved manual teach-back work.

Foundation confirmed:

- Hotkey Clipboard_v0.1 works.
- Correction Engine_v0.2 works.
- Correction Engine_v0.3 structured result works.
- Manual Correction Capture_v0.1 works.
- Correction Event Integration_v0.1 works.
- SQLite event saving works for dictionary and manual events.
- Recent event inspection works.
- pytest works when pytest.ini exists.